



Міністерство освіти і науки України

КПІ ім. Ігоря Сікорського

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

**Лабораторна робота № 3**

З дисципліни “Сучасні технології розробки WEB-застосувань на платформі  
Microsoft.Net ”

студентки III курсу ФІОТ

групи ІК-12

**Макарчук Ольги**

Перевірив:  
Бардін В.

Київ 2023

**Мета:**

1. Ознайомитися з основами створення REST веб-API та методологією С4 для відображення архітектури системи.
2. Ознайомитися з основами створення ER-діаграм для представлення структури бази даних.

**Завдання:**

1. З дотриманням вимог REST-у спроектувати веб-API для обраної(згідно варіанту) доменної області, використовуючи методологію С4 для створення діаграми архітектури системи.
2. Створити ER-діаграму для DAL (Data Access Layer), яка відображатиме структуру бази даних веб-API.
3. Оформити спроектоване рішення у вигляді звіту до лабораторної роботи.

**Варіант 3:**

|   |                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 | Театральна каса.<br>Розповсюдження<br>квитків | <ol style="list-style-type: none"><li>1. Афіша вистав надає функції пошуку потрібної вистави за автором, назвою, жанром та датою.</li><li>2. Кожній виставі відповідає кількість квитків різної вартості.</li><li>3. Квитки можна безпосередньо продати чи попередньо забронювати та згодом перевести їх у стан проданих.</li></ol> <p><b>Функціональні вимоги:</b></p> <ol style="list-style-type: none"><li>1. Ведення афіши вистав;</li><li>2. Продаж квитків</li></ol> |
|---|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## ER-діаграма



### Таблиця: **Genre**

Поля:

- IdGenre: int – унікальний ідентифікатор жанру
- NameGenre: varchar(20) – назва жанру

### Таблиця: **Author**

Поля:

- IdAuthor: int – унікальний ідентифікатор автора
- FirstName: varchar(20) – ім'я автора
- MiddleName: varchar(20) – по батькові автора
- LastName: varchar(20) – прізвище автора

### Таблиця: **Performance**

Поля:

- IdPerformance: int – унікальний ідентифікатор вистави

- Name: varchar(20) – назва вистави
- IdGenre: GUID – унікальний ідентифікатор жанру
- IdAuthor: GUID – унікальний ідентифікатор автора
- DateTimeEvent: datetime – дата та час вистави

#### Таблиця: **TypeOfTickets**

Поля:

- IdType: int – унікальний ідентифікатор типу квитків
- NameGenre: varchar(20) – назва типу квитків

#### Таблиця: **AllTickets**

Поля:

- IdAllTickets: int – унікальний ідентифікатор квитків
- IdType: int – унікальний ідентифікатор типу квитків
- IdPerformance: int – унікальний ідентифікатор вистави
- CountOfTickets: int – кількість квитків
- Price: int – ціна квитка

#### Таблиця: **StatusTicket**

Поля:

- IdStatus: int – унікальний ідентифікатор статусу квитка
- NameStatus: varchar(20) – назва статусу квитка

#### Таблиця: **Ticket**

Поля:

- IdTicket: int – унікальний ідентифікатор квитка
- IdAllTickets: int – унікальний ідентифікатор квитків
- IdUser :int - унікальний ідентифікатор користувача
- PhoneNumber: varchar(20) – номер телефону
- IdStatus: int - унікальний ідентифікатор статусу квитка
- SeatNumber: int – номер місця

### Таблиця: **Role**

Поля:

- IdRole: int – унікальний ідентифікатор ролі користувача
- NameRole: varchar(20) – назва статусу ролі користувача

### Таблиця: **User**

Поля:

- IdUser: int – унікальний ідентифікатор статусу квитка
- Email: varchar(20) – електронна пошта користувача
- PasswordHash: varchar(20) – хеш пароля
- FirstName: varchar(20) – Ім'я
- MiddleName: varchar(20) – По батькові
- LastName: varchar(20) – Прізвище
- IdRole: int - унікальний ідентифікатор ролі користувача

### Таблиця: **RefreshToken**

Поля:

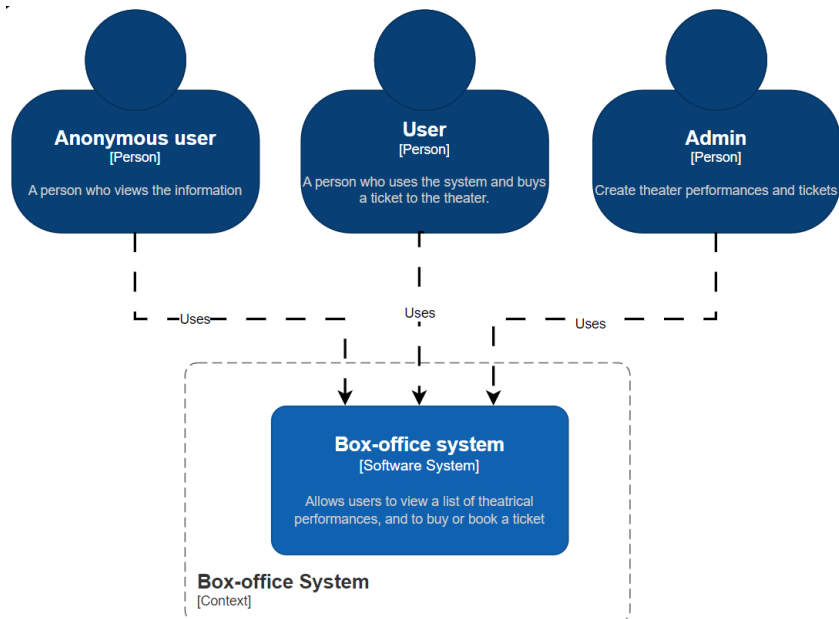
- IdToken: int – унікальний ідентифікатор ролі користувача
- Expires: varchar(20) – назва статусу ролі користувача
- IdUser: int – унікальний ідентифікатор користувача

## Діаграма C4

**C4 (Context, Container, Component, Code)** - це методологія для моделювання архітектури програмного забезпечення на різних рівнях абстракції. Ця методологія дозволяє структурувати розгляд архітектури на різних рівнях деталей, починаючи від високорівневого контексту та закінчуючи кодом компонентів.

### Рівень 1 : System Context Diagram

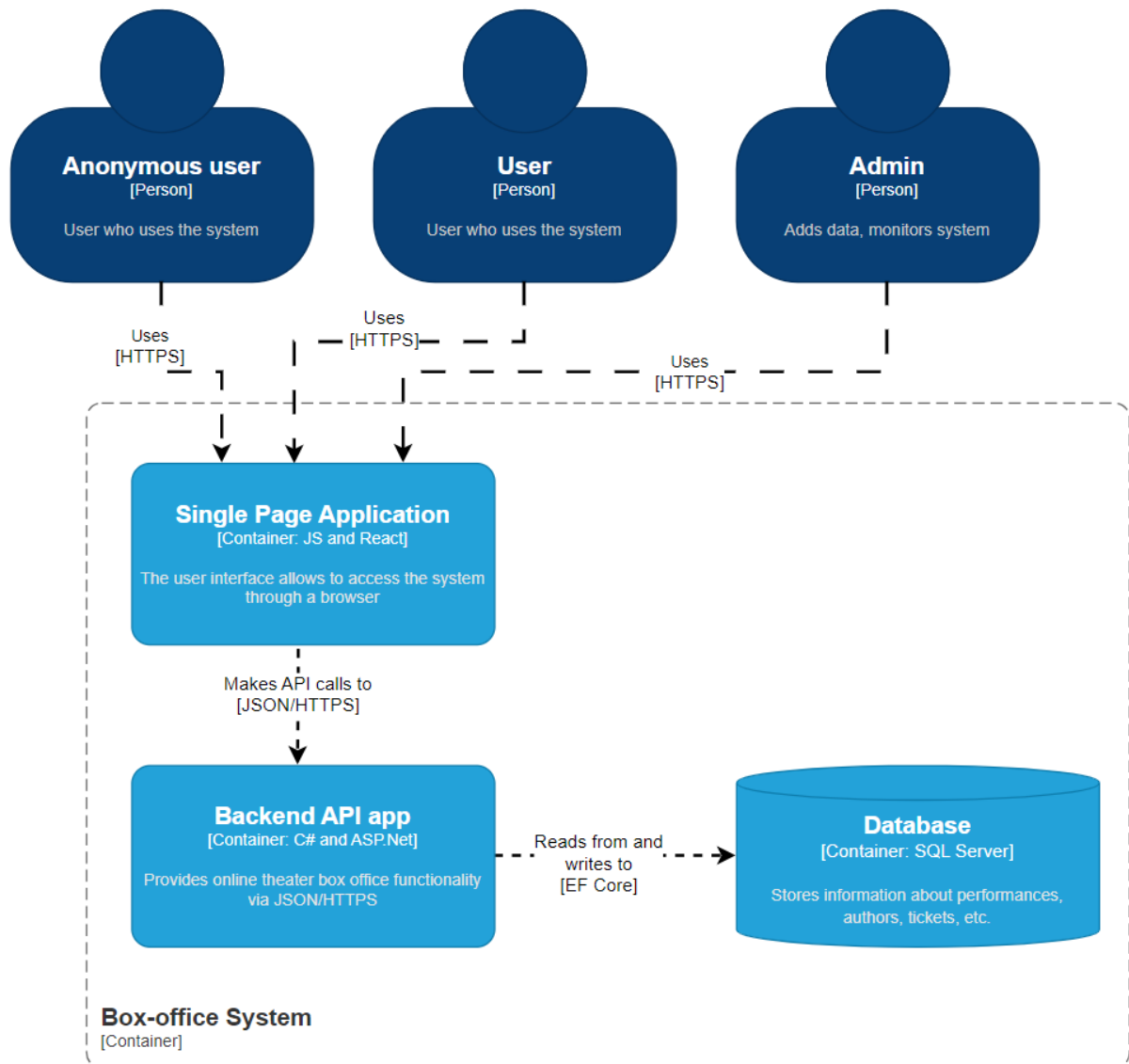
System Context Diagram (Контекстна діаграма системи) – Це перший рівень діаграм C4 і представляє високорівневий огляд системи та її зовнішнього середовища. Вона вказує на основні актори (людей або системи), які взаємодіють з системою, і визначає основні взаємодії між ними.



Система театральної каси має один тип користувача, який є неавторизованим. Користувач може переглядати театральну афішу, де є функції перегляду всіх вистав, пошуку вистав по назві, автору та даті. Система надає можливість, що дозволяє обрати категорію та номер місця, а також купувати та бронювати квитки, які згодом можна перевести у стан проданих.

## Рівень 2 : Container Diagram

Container Diagram(Діаграма контейнерів): На цьому рівні архітектурного моделювання, система розглядається як набір контейнерів (наприклад, веб-додаток, база даних, мобільний додаток тощо). Це дозволяє деталізувати внутрішню структуру системи та показати взаємодії між контейнерами.

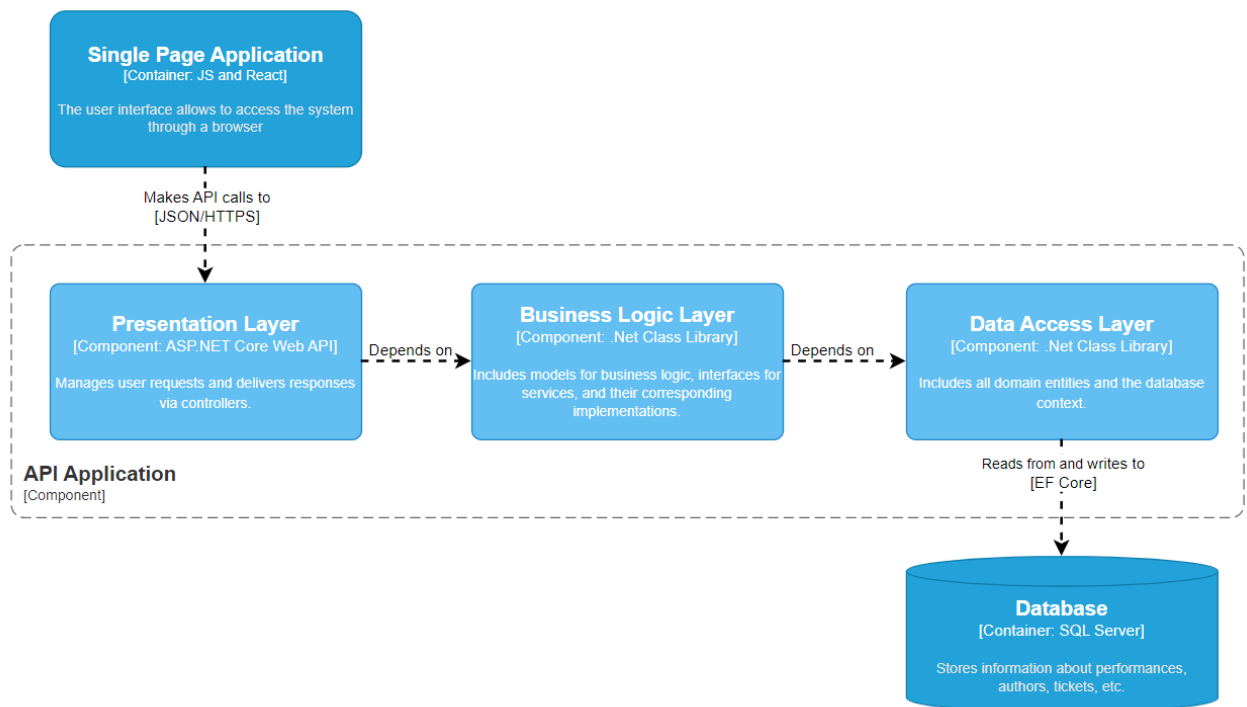


Система театральної каси поділяється на три компоненти:

1. Веб-застосунок, через який користувач взаємодіє з системою.
2. Серверний застосунок, де виконуються запити користувача. Функціонал реалізується через REST API.
3. База даних, яка зберігає всю необхідну інформацію, яка задіяна у системі.

### Рівень 3 : Component Diagram

Component Diagram (Діаграма компонентів) - на цьому рівні система розглядається як набір компонентів, які визначають внутрішню реалізацію контейнерів. Компоненти в цьому контексті можуть бути реальними програмними елементами, такими як класи, бібліотеки, модулі або підсистеми.



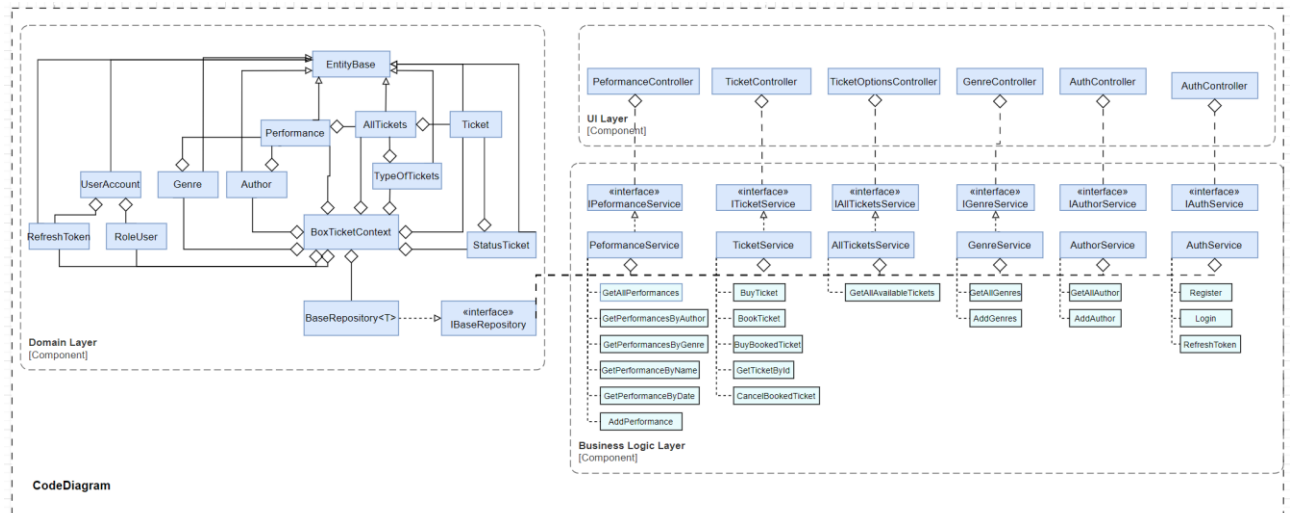
На етапі розробки застосунку буде використано N-layer архітектуру, а вона вже розділяє застосунок на три шара:

1. User Interface Layer (Шар представлення) – на цьому рівні відбувається взаємодія користувача, виконання запитів, та представлення необхідної інформації за допомогою графічного інтерфейсу. За приймання запитів відповідають контролери.
2. Business Logic Layer (Шар бізнес-логіки) – на цьому рівні відбувається обробка даних та реалізація логіки операцій і функцій.
3. Data Access Layer (Шар доступу до даних) – на цьому рівні відбувається взаємодія з базою.



## Рівень 4 : Code Diagram

Code Diagram(Діаграма коду) - на останньому рівні представлення архітектури використовується діаграма коду. Вона може включати в себе високорівневі структури коду, залежності між класами або компонентами, та інші деталі реалізації.



DLL:

- `BoxTicketContext` – контекст бази даних;
- `Performance`, `Genre`, `Author`, `AllTickets`, `TypeOfTickets`, `Ticket`, `StatusTicket`... – сутності бази даних;
- `IBaseRepository<T>` – реалізація `Repository` з generic інтерфейсом;
- `BaseRepository<T>` – реалізація generic класу для репозиторіїв;

BLL:

- `IPerformanceService`, `ITicketService`, `IAllTicketsService`... – інтерфейси сервісів;
- `PerformanceService`, `TicketService`, `AllTicketsService`... – імплементація сервісів;

Також на діаграмі зображено основні функції кожного сервіса.

UIL:

- `PerformanceController`, `TicketController`, `AllTicketController`... – контролери;

## Кінцеві точки REST API(Endpoints)

| Auth          |                                     |  | ^   |
|---------------|-------------------------------------|--|-----|
| POST          | /api/Auth/register                  |  | ✓   |
| POST          | /api/Auth/login                     |  | ✓   |
| GET           | /api/Auth/refresh-token             |  | ✓ 🔒 |
| Author        |                                     |  | ^   |
| GET           | /api/Author/GetAll                  |  | ✓   |
| POST          | /api/Author/Add                     |  | ✓ 🔒 |
| Genre         |                                     |  | ^   |
| GET           | /api/Genre/GetAll                   |  | ✓   |
| POST          | /api/Genre/Add                      |  | ✓ 🔒 |
| Performance   |                                     |  | ^   |
| POST          | /api/Performance/GetByDate          |  | ✓   |
| POST          | /api/Performance/GetByAuthor        |  | ✓   |
| POST          | /api/Performance/GetByGenre         |  | ✓   |
| POST          | /api/Performance/GetByName          |  | ✓   |
| GET           | /api/Performance/GetAll             |  | ✓   |
| POST          | /api/Performance/Add                |  | ✓ 🔒 |
| Ticket        |                                     |  | ^   |
| POST          | /api/Ticket/Book                    |  | ✓ 🔒 |
| POST          | /api/Ticket/Buy                     |  | ✓ 🔒 |
| PUT           | /api/Ticket/BuyBooked               |  | ✓ 🔒 |
| GET           | /api/Ticket/MyTickets               |  | ✓ 🔒 |
| DELETE        | /api/Ticket/CancelBookedTicket      |  | ✓ 🔒 |
| TicketOptions |                                     |  | ^   |
| POST          | /api/TicketOptions/AvailableTickets |  | ✓   |